TEMPORAL WORKFLOW DESIGN

Fulfillment Sync

Getting an approved sales order to the warehouse without waiting for a scheduled data copy, and knowing whether it arrived.

Describes the code in this repository. It runs on local simulators with synthetic data and has not been run against Oracle Fusion or Palantir Foundry.

1. The problem

After an order is approved in Oracle Fusion, warehouse staff can't see it in Fulfillment Manager for about 15–20 minutes. Oracle accepts the order immediately, then builds shipment lines in the background. Fulfillment Manager reads from Palantir Foundry, and those lines reach Foundry only through a scheduled CData snapshot of the whole table. The app receives no signal from Oracle at any point.

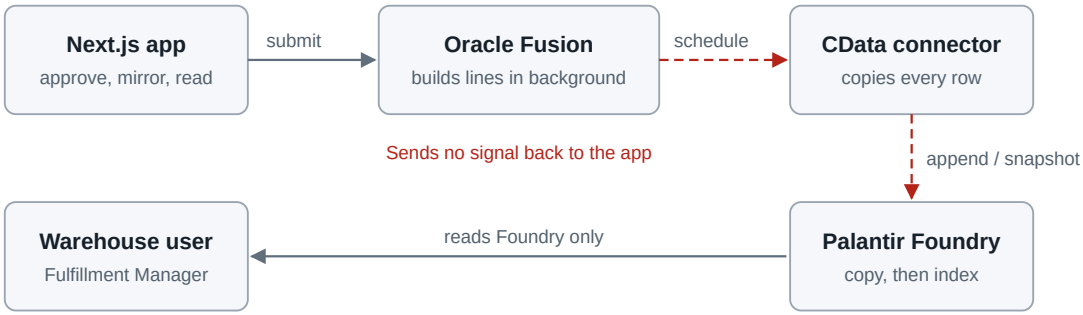


Figure 1. Today's path. Dashed lines run on a schedule.

Delay = Oracle processing + wait for next snapshot + snapshot run time + indexing

The client raised these issues on the architecture call:

Issue	What it means
Delay	Staff wait for the next snapshot, even when Oracle finished long ago.
Growth	Each snapshot reads everything, so it slows as data grows and uses the shared Oracle REST limit.
Silent failure	An order that doesn't reach fulfillment raises nothing. Finding out means a manual check in Oracle.
Constraints	Oracle populates too many fields to write shipment lines optimistically. UIs can't read Oracle live because of REST limits and paging. Analytics still need the snapshot.

This reproduces locally with no Temporal running. With the app and simulators in Docker, 5 seconds of Oracle processing and a 60-second snapshot, a browser test saw Oracle holding the line after 5.9 seconds, while the order reached Fulfillment Manager only after 64.1 seconds. On simulated time with a 15-minute schedule, the slowest order took 17.5 minutes with no existing data and 22.3 minutes with 100,000 lines. The schedules come from the client, but the other timings are assumptions, so these aren't Shift4 measurements.

2. The proposed solution

One durable Temporal workflow follows each approved order and **reconciles** it. Once Oracle's shipment lines cover the order, one operation reads both systems, writes only what differs, and repeats until the shipment lines and the fulfillment order the warehouse reads both match. That single comparison handles retries, restarts, late changes and verification, and it also answers "why isn't this order here?" without opening Oracle. A scheduled sweep finds any approved order that was missed, and a scheduled audit corrects drift on delivered ones. The existing snapshot keeps running for analytics and correction, and could later run less often. Release for picking, create shipment, carriers and invoices aren't part of this phase.

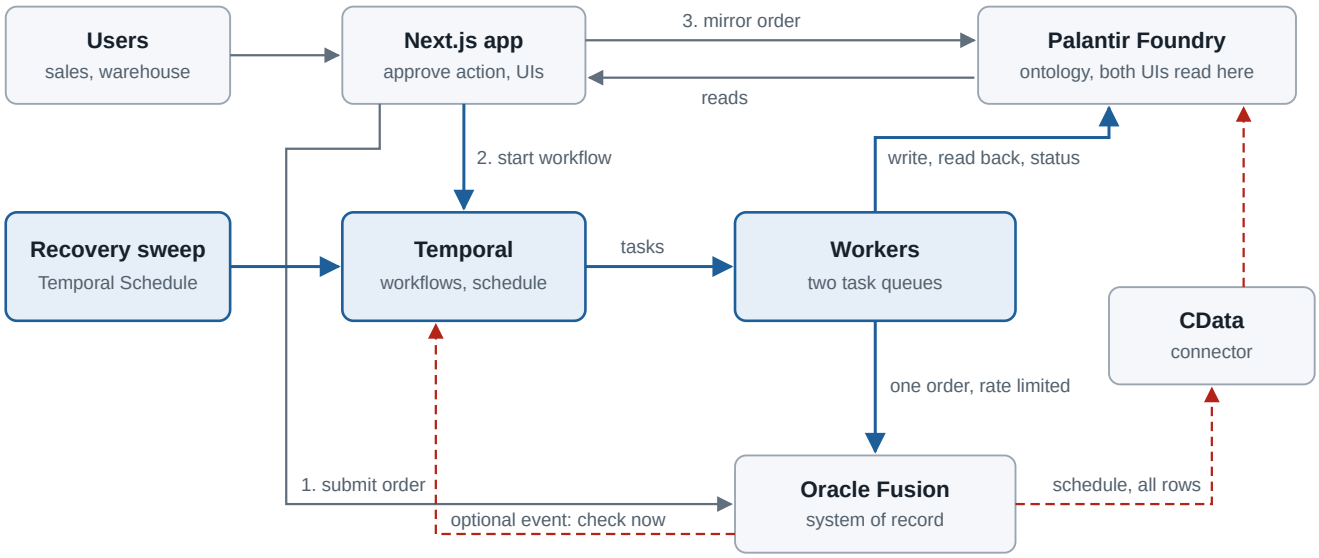


Figure 2. Blue marks what this design adds. Dashed lines are scheduled or optional. Workers reach Oracle and Foundry only through two adapters, which the client replaces with their own helpers. Oracle Fusion, Foundry and CData are simulated in this repository.

Issue	How the solution answers it
Delay	The order no longer waits for a schedule. It appears soon after Oracle creates its lines, so Oracle's own processing time is the floor.
Growth	Each read covers one order, so its cost doesn't depend on the size of the table. Oracle reads share a limit across workers.
Silent failure	Every order ends with a known status and reason, overdue orders raise an alert event, and one command explains any order from live data.
Lost responses, restarts, repeats	Workflow ID is known before any Oracle call, state survives worker restarts, and each pass reads fresh data and writes only differences, so repeating it is safe.
Customer data	Rows stay inside the activities, so Temporal history holds only keys, quantities and counts.
Rollback	`FULFILLMENT_SYNC` off returns the app to today's behaviour.

Measured through the real app flow, on the same simulators and settings. Oracle processing (90 seconds), snapshot costs and the 120 calls per minute limit are assumptions, so these show how each option behaves and aren't Shift4 measurements.

Path	Slowest, no existing lines	Slowest, 25,000 lines	Slowest, 100,000 lines	Connector calls per hour at 100,000
Snapshot every 15 min	17.5 min	18.8 min	22.3 min	673
Snapshot every 5 min	7.5 min	8.8 min	12.3 min	2,152
Snapshot every 1 min	3.5 min	4.8 min	8.3 min	6,816
Incremental every 5 min	7.5 min	7.5 min	7.5 min	11
This workflow	Not measured	**1.8 min**	Not measured	8 calls for that order

A faster snapshot cuts the wait but multiplies load on the shared Oracle limit, and still slows with data. An incremental connector stays flat and cheap, which makes it a good partner for the snapshot, but every order still waits for a schedule and gets no status of its own. The snapshot rows use simulated time, and the workflow row used real time with a real Temporal server, once at 25,000 lines.

3. Workflow design

`deliverOrderToFulfillment`: one per approved order

The approve action submits to Oracle, starts this workflow, then mirrors to Foundry, so a mirror failure can't hold the order back. The workflow file holds only the delivery path, with its Temporal plumbing in a separate runtime file, and all I/O happens in activities. Its ID, `order-delivery:<SourceTransactionNumber>`, is known before any Oracle call, so a lost response can't lose the order. `USE_EXISTING` makes a double click join the running workflow. `ALLOW_DUPLICATE` lets the sweep or a retry start again after a run has closed.

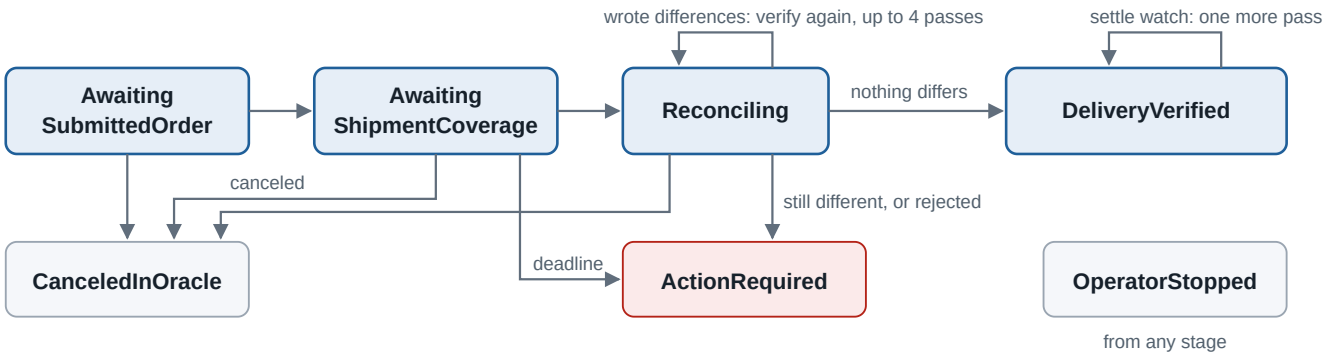


Figure 3. Stages, held in the `DeliveryStage` search attribute. `DeliveryOverdue` is a separate flag raised by a timer, so an overdue order still shows where it is, and work continues.

Activities

The first three are the required core. The sweep and audit are optional.

Activity	What it does	Uses	Queue and retry
`checkOrderReady`	One check of how far Oracle has got. First it fails until Oracle returns the order as submitted, which covers a lost response. Then it fails until active shipment lines cover every quantity, across all pages and warehouses, re-reading the order every fifth attempt to notice a cancellation.	Oracle: `findOrder`, `listShipmentLines`	`oracle-reads`. Polling, until the deadline
`reconcileOrder`	The core. Reads the order, Oracle's lines and Foundry's objects, builds the expected fulfillment orders, writes what's missing, older or changed, and removes what Oracle no longer supports. Returns only counts.	Oracle: both reads. Foundry: read, `sync-shipment-lines`, `remove-shipment-lines`	`oracle-reads`. 30-minute window
`recordDeliveryStatus`	Sends an audit or alert event, and saves the stage on the sales order. A failure here never changes the result.	Event sink. Foundry: `record-fulfillment-sync`	`order-delivery`. 3 attempts
`sweepMissedOrders`	Starts recent submitted orders with no final status. Skips running and stopped ones, and stops after 5 failed runs per order.	Recovery index. Temporal client	`order-delivery`. 3 attempts

`recoverySweepWorkflow` and `deliveryAuditWorkflow`: the optional safety nets

A Temporal Schedule runs it every 15 minutes, looking back 24 hours and covering at most 100 orders. It catches what a single request can't: Temporal unavailable at approval, or a process that crashed between submit and start. Overlap is set to Skip, so a slow sweep can't pile up. Without it, the snapshot remains the fallback. A second Schedule runs the audit daily through `auditDeliveredOrders`, which runs recently delivered orders through reconcile again to correct drift after their workflows ended. Both run below the priority of new approvals.

Temporal features used

Feature	Use
Activity retries as polling	From 5 seconds, growing 1.5x to 60 seconds. No history growth, no held worker slot, and the UI shows attempts and the last result.
Timeouts	30 seconds per attempt, a 4-hour deadline for finding and waiting, and a 30-minute window that rides out a Foundry outage.
Failure types	A 429 honours `Retry-After`. A 401 or 403 keeps retrying, so an expired credential doesn't fail every open order. A rejected request or unexpected error stops at once.
Task queue rate limit	Holds Oracle-reading activities to one limit across all workers.
Durable timers	Raise the overdue flag, and run the settle watch that catches a change made soon after delivery.
Signal, Signal-with-Start, batch operations	An operator or Oracle event wakes a waiting order to check at once, or starts one that's missing. After an outage, one command signals every overdue order. It's only a hint, so a lost event only costs time.
Task Queue Priority	Recovery and audit work run below new approvals, so a backlog can't delay them.
Search attributes, query	`DeliveryStage`, `DeliveryOverdue` and `SourceOrderNumber` find orders by situation. `getDeliveryProgress` returns the current state, and each run shows a summary and live status in the Temporal UI.
Schedules, cancellation	Schedules run the sweep and audit, and an operator can stop a run cleanly.

Feature	Use
Replay and time-skipping tests	One replays recorded history to catch breaking code changes. The other proves the real 30-minute and 4-hour timings in seconds.
SDK and workflow metrics	Prometheus metrics for Oracle's real wait, approve to visible and outcomes, which turn the first DEV run into measurements. Rising schedule-to-start latency shows workers falling behind.

Failures and edge cases

What goes wrong	What happens
Oracle rejects the submit	No workflow starts, and the user sees the error. A 429 tells the user Oracle is busy.
Submit response lost, or a 5xx	Workflow starts as `uncertain` and trusts what Oracle holds.
Submit never reached Oracle	Keeps checking, then ActionRequired. Never a false success.
Temporal unavailable at approval	Approval succeeds, `NotStarted` is recorded, and the sweep starts it later.
Foundry mirror fails after submit	Workflow has already started, so the order is still delivered.
Worker restarts during a wait	Another worker continues from history, with no duplicates.
Foundry outage, or a lost write response	Retries, and the next read shows what was really written, so nothing is written twice.
Oracle changes between read and write	The next pass sees Foundry is older and writes the current rows.
Same timestamp, different data	Content is compared too, so the row is written.
Line added, or order canceled, during the run	Line added: delivery waits until it's covered. Canceled: ends as CanceledInOracle, including just before reconciling.
Lines written, but the fulfillment order missing or wrong	Not delivered: the next pass writes it, and verifies it.
Row in Foundry that Oracle no longer has	Removed, and a later snapshot doesn't bring it back. An empty Oracle answer never removes anything.
Foundry stale-object conflict (409)	Retries, and a fresh read resolves it.
Expired credentials	Keeps retrying, and recovers once they're fixed.
A bad date or programming error	Stops at once with the reason, rather than looking like a long wait.
Lines never appear	Overdue at 30 minutes, then ActionRequired at 4 hours, with the reason recorded.
An order keeps failing	The sweep stops restarting it after 5 failed runs, and never restarts a delivered or stopped order. One order that can't start doesn't affect the others.
Oracle changes after delivery	Caught by the settle watch soon after delivery, then by the snapshot or daily audit.

4. Pros and cons

Pros

Benefit	Why it matters
No wait for a schedule	The order appears soon after Oracle creates its lines. With the same containers and settings, approve to visible went from 64.1 seconds to 7.9 seconds, which is Oracle's 5 seconds plus the wait for the next check.
Timing no longer depends on luck	On the simulators, the wait varied from 5.5 to 20.5 seconds with the moment of approval today, and stayed at 2.1–2.2 seconds with the workflow.
Cost follows the order, not the table	Each read covers one order, so this path doesn't depend on table size. It used 8 Oracle calls for its order at 25,000 lines.
One simple, repeatable core	Retries, restarts, late changes and verification all go through one comparison, and repeating it is always safe.
Every order has an answer	Each ends with a known status and reason, overdue orders raise an event, and `make explain` answers from live data with no manual check in Oracle.
Delivered means the warehouse can see it	The status is set only when a pass finds the lines and the fulfillment order match, and the settle watch checks once more.
Stays correct after delivery	A low-priority daily audit corrects drift, so a mistake doesn't wait for someone to notice it.
Real numbers from the first DEV run	Every order records Oracle's wait and approve to visible, which answers the biggest unknown and shows the improvement.
Customer data stays out of Temporal	History holds keys, quantities and counts, so no Payload Codec is needed for this and large orders can't hit payload limits.
Survives failures	Worker restarts, lost responses, Foundry outages and expired credentials pause the work rather than lose it.
Operators can act	One command retries an order, and one signals all overdue orders after an outage.
Ready for Oracle events	If Oracle's events cover this status, they cut the wait with no redesign, and polling remains the fallback.
Low-risk rollout	One flag returns the app to today's behaviour, the sweep is optional, and the snapshot remains the safety net.

Cons

Drawback	Effect	How to handle it
Not run on real systems	Adapters, Foundry actions and all timings are still assumptions.	Implement the adapters on the client's DEV environment
Oracle time is the floor, and unknown	The workflow can't read lines Oracle hasn't created, so no latency can be promised yet.	Measure each step on DEV
Two writers to the same data	Foundry compares the edit's time with Oracle's UTC timestamp, so a clock difference could hide a change from the snapshot. The settle watch catches this only soon after delivery. These rules come from Foundry's documentation and aren't tested on a real tenant.	Confirm timestamp type and clock difference, and test on DEV

Drawback	Effect	How to handle it
The fulfillment object is our model	Lines and fulfillment orders are written and verified, but the real object's name, keys and fields are unconfirmed, and they must match the client's transform.	Get them from the client, then adjust one function and the adapter
Rows deleted after the workflow ends	Foundry keeps an object an Action created when its source row disappears. The audit removes it, but only for orders inside its lookback.	Learn whether Oracle deletes or only cancels shipment lines
More Oracle reads per order	Each pass makes at least two Oracle calls, and the audit adds more. The limit counts activities, not HTTP calls, the app or the connector. A stricter limit also adds delay: 1 activity per second is part of the time above.	Get the real limit, then tune the rate and polling
New parts to run	Temporal and workers need operating, and the team needs to learn them.	Start on EC2 or Temporal Cloud at low volume
Integration work	The client implements two adapters with six methods, creates three Foundry actions, and adds a service identity. A per-row version check may need a function-backed action.	Use the simulators as a reference, and skip the sweep at first
Alerts need a destination	Overdue and ActionRequired events can post to a webhook, but no channel is chosen yet.	Choose one with the client
Oracle events aren't connected	The check-now path is built and tested, but nobody has confirmed which Oracle status means shipment lines exist.	Confirm with the client, or keep polling
Holds aren't recognised	An order on hold polls until the deadline, then shows ActionRequired.	Learn how holds appear in Oracle
Not production hardened	No workflow versioning yet, and the worker bundles its code at startup. Priority's effect under real backlog isn't measured. History lasts 30–90 days unless exported through the event sink.	Add versioning and a prebuilt bundle before the first release